



MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SYSTEMS ENGINEERING

AI Project CEP

AI Minesweeper Game Project

Names w/ Roll No.:

Lachhman Singh (21CS016)

Shahwaiz Qureshi (21CS090)

M. Mashaal Zulfikar (21CS105)

Batch: **21CS**

Date: **21-04-2025**

Submitted to: **Dr. Sammer Zai**

Department of Computer Systems Engineering Mehran University of Engineering and Technology, Jamshoro			
Course: Artificial Intelligence (CS-452)			
Instructor	Dr. Sammer Zai	Assignment Type	CEP based Project
Semester	7 th	Year	7th
Batch	21CS	Section	I & II
Submission Deadline	11-04-2025	Assessment Score	10

Problem Description
Design and develop an intelligent agent that addresses a complex real-world problem using Artificial Intelligence techniques. Your agent should be capable of perception, decision-making, and action execution in its respective environment. Task Guidelines: Problem Selection: Identify a real-world problem where an intelligent agent can be deployed (e.g., medical diagnosis, smart traffic management, automated customer service, game AI, or personalized recommendation systems). Agent Design: Define the structure, functionalities, and environment in which the agent will operate. Decision-Making Mechanism: Use AI techniques for decision-making. Implementation: Develop a working prototype using any suitable programming language (e.g., Python, Java). Evaluation: Test and validate your agent's performance using suitable metrics. Report & Presentation: Submit a detailed technical report and present your work in class.

Note: Submit your CEP based assignment in hard form. The assignment will be assessed against the rubrics (given on the next page) which provide clear assessment criteria to maintain fairness and clarity.

Department of Computer Systems Engineering Mehran University of Engineering and Technology, Jamshoro			
Course: Artificial Intelligence (CS-452)			
Student Name		Roll Number	CEP based Project
Batch	21CS	Section	I & II

Assessment Rubrics

S#	Assessment Rubrics	Assessment				
		Excellent (10-9)	Good (8-6)	Satisfactory (5-3)	Needs Improvement (2-0)	Total Score (10)
R1	Problem Definition & Relevance	Clearly defines a real-world complex problem with strong justification.	Defines a relevant problem but lacks some justification.	Problem is somewhat relevant but lacks depth.	Problem is not clearly defined or lacks relevance.	
R2	Agent Design & Architecture	Well-structured agent design with a clear flow of perception, decision-making, and actions.	Design is logical but lacks detailed explanation.	Basic agent design with minimal explanation.	Poorly designed or incomplete architecture.	
R3	AI Technique Used	Uses an appropriate AI technique with proper justification and implementation.	Uses an AI technique but lacks strong justification.	Uses an AI technique but implementation is weak.	No proper AI technique used.	
R4	Implementation & Functionality	Fully functional prototype with proper working mechanisms.	Mostly functional with minor issues.	Partially functional but not fully tested.	Implementation is incomplete or non-functional.	
R5	Evaluation & Testing	Proper evaluation using relevant performance metrics.	Evaluation performed but lacks some depth.	Limited testing with minimal evaluation.	No proper testing or evaluation performed.	
R6	Report & Presentation	Well-structured report with clear explanations and a professional presentation.	Report and presentation are good but lack some depth.	Basic report with minimal presentation effort.	Report and presentation are incomplete or unclear.	
Cumulative Marks (out of 60)						
Total Marks out of 10 = Cumulative Marks/60 *10						



21CS016, 090, 105 AI Project

CEP 7th Semester

AI Minesweeper Game Project

<u>Student Name</u>	<u>Roll No.</u>
Lachhman Singh	21CS016
Shahwaiz Qureshi	21CS090
Mashaal Zulfikar	21CS105
Submitted to:	Dr. Sammer Zai

Background

This project was developed with the help of the course materials of "CS50's Introduction to Artificial Intelligence with Python." This project combines strategy and logic, allowing to apply AI concepts like decision-making and risk assessment. It serves as a practical learning experience and a way to explore real-world applications of AI while contributing to the open-source community.



This screenshot is taken from Lecture 1: Knowledge → [Minesweeper - CS50's Introduction to Artificial Intelligence with Python](#). This shows the demonstration of how Minesweeper game works with its AI techniques applied.

Introduction

Minesweeper is a classic puzzle game that challenges players to uncover a grid of squares while avoiding hidden mines. The game requires strategic thinking, logical reasoning, and risk assessment, making it an example for the application of artificial intelligence. This project aims to develop an intelligent agent capable of playing Minesweeper autonomously, with AI techniques to make informed decisions based on the game state.

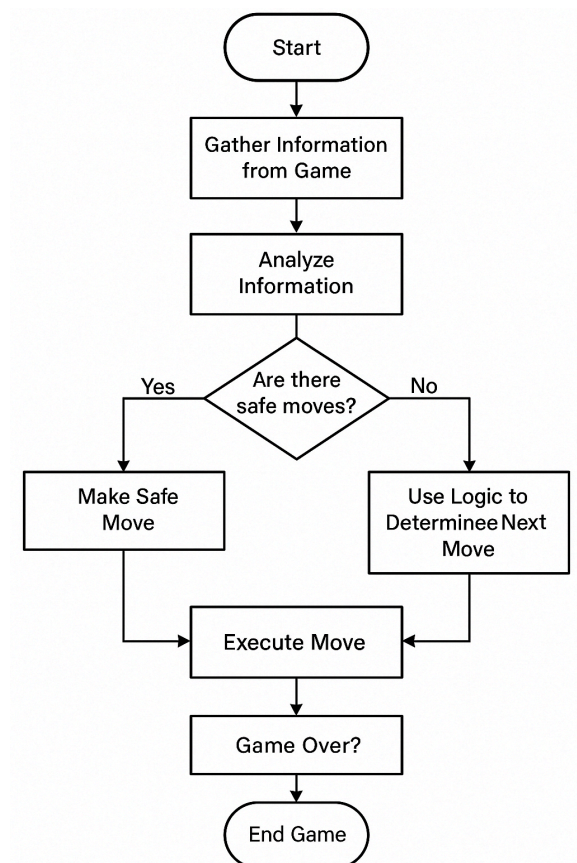
Problem Statement

The main goal of this project is to create an AI agent that can play Minesweeper. This agent will be able to understand the game environment, make smart decisions based on the information it has, and take actions to increase its chances of winning. The need for this project comes from the growing interest in artificial intelligence and how it can solve complex problems.

By using concepts from "CS50's Introduction to Artificial Intelligence with Python," this project helps improve our understanding of AI while showing how it can be applied in a fun and engaging way.

The project gains deeper insights into game AI. It establishes a foundation for exploring AI applications in more complex scenarios, highlighting the potential of intelligent agents in real-world situations.

Game Design



Flowchart of Minesweeper Game:

The flowchart shows how the AI agent makes decisions while playing Minesweeper. It starts when the agent is initialized. First, the agent gathers information by looking at the game grid to see which squares are revealed and how many mines are nearby. Next, it analyzes this information to figure out possible moves and checks if there are any safe options. If there are safe moves, the agent will reveal one of those squares. If not, it uses logic to decide on the best move to make. After making a move, the agent checks if the game is over, either by hitting a mine or clearing the board. If the game is finished, it records the result; if not, it goes back to gathering information to keep playing. This process shows how the agent adapts and makes strategic choices based on the current state of the game.

Project Overview

The goal of this project is to create an AI agent capable of playing the classic game of Minesweeper. The AI is designed to make informed decisions based on the game state, using logical reasoning to deduce safe moves and identify mines.

The project is organized around three main classes:

- **Minesweeper:** Manages the game board, including mine placement and game logic.
- **Sentence:** Represents logical statements about the game, enabling the AI to reason about the board state.
- **MinesweeperAI:** The AI player that interacts with the game, making decisions based on its knowledge and the current game state.

```
import itertools
import random
import re

class Minesweeper:
    """
    Minesweeper game representation
    """

    def __init__(self, height=8, width=8, mines=8):
        # Set initial width, height, and number of mines
        self.height = height
        self.width = width
        self.mines = set()

        # Initialize an empty field with no mines
        self.board = []
        for _ in range(self.height):
            row = [False for _ in range(self.width)]
            self.board.append(row)

        # Add mines randomly
        while len(self.mines) != mines:
            i = random.randrange(height)
            j = random.randrange(width)
            if not self.board[i][j]:
                self.mines.add((i, j))
                self.board[i][j] = True
```

```

# At first, player has found no mines
self.mines_found = set()

def print(self):
    """
    Prints a text-based representation
    of where mines are located.
    """
    for i in range(self.height):
        print("--" * self.width + "-")
        for j in range(self.width):
            if self.board[i][j]:
                print("|X", end="")
            else:
                print("| ", end="")
        print("|")
        print("--" * self.width + "-")

def is_mine(self, cell):
    i, j = cell
    return self.board[i][j]

def nearby_mines(self, cell):
    """
    Returns the number of mines that are
    within one row and column of a given cell,
    not including the cell itself.
    """

    # Keep count of nearby mines
    count = 0

    # Loop over all cells within one row and column
    for i, j in itertools.product(range(cell[0] - 1, cell[0] + 2), range(cell[1] - 1, c
        # Ignore the cell itself
        if (i, j) == cell:
            continue

```



```

        # Update count if cell in bounds and is mine
        if 0 <= i < self.height and 0 <= j < self.width and self.board[i][j]:
            count += 1

    return count

def won(self):
    """
    Checks if all mines have been flagged.
    """
    return self.mines_found == self.mines

class Sentence:
    """
    Logical statement about a Minesweeper game
    A sentence consists of a set of board cells,
    and a count of the number of those cells which are mines.
    """

    def __init__(self, cells, count):
        self.cells = set(cells)
        self.count = count

    def __eq__(self, other):
        return self.cells == other.cells and self.count == other.count

    def __str__(self):
        return f"{self.cells} = {self.count}"

    def known_mines(self):
        """
        Returns the set of all cells in self.cells known to be mines.
        """
        known_safes = self.known_safes() or set()
        remaining_cells = self.cells - known_safes
        return remaining_cells.copy() if len(remaining_cells) <= self.count else se

```

```

def known_safes(self):
    """
    Returns the set of all cells in self.cells known to be safe.
    """
    return self.cells.copy() if self.count <= 0 else set()

def mark_mine(self, cell):
    """
    Updates internal knowledge representation given the fact that
    a cell is known to be a mine.
    """
    if cell in self.cells:
        self.cells.discard(cell)
        self.count -= 1

def mark_safe(self, cell):
    """
    Updates internal knowledge representation given the fact that
    a cell is known to be safe.
    """
    if cell in self.cells:
        self.cells.discard(cell)

class MinesweeperAI:
    """
    Minesweeper game player
    """

    def __init__(self, height=8, width=8):
        # Set initial height and width
        self.height = height
        self.width = width

        # Keep track of which cells have been clicked on
        self.moves_made = set()

        # Keep track of cells known to be safe or mines

```

```

self.mines = set()
self.safes = set()

# List of sentences about the game known to be true
self.knowledge = []

def mark_mine(self, cell):
    """
    Marks a cell as a mine, and updates all knowledge
    to mark that cell as a mine as well.
    """
    self.mines.add(cell)
    for sentence in self.knowledge:
        sentence.mark_mine(cell)

def mark_safe(self, cell):
    """
    Marks a cell as safe, and updates all knowledge
    to mark that cell as safe as well.
    """
    self.safes.add(cell)
    for sentence in self.knowledge:
        sentence.mark_safe(cell)

def add_knowledge(self, cell, count):
    """
    Called when the Minesweeper board tells us, for a given
    safe cell, how many neighboring cells have mines in them.

    This function should:
    1) mark the cell as a move that has been made
    2) mark the cell as safe
    3) add a new sentence to the AI's knowledge base
       based on the value of `cell` and `count`
    4) mark any additional cells as safe or as mines
       if it can be concluded based on the AI's knowledge base
    5) add any new sentences to the AI's knowledge base

```

```

        if they can be inferred from existing knowledge
        """

        self.moves_made.add(cell)
        neighbors = self.neighboring_cells(cell)
        sentence = Sentence([cell, *neighbors], count)

        self.knowledge.append(sentence)
        self.mark_safe(cell)

    for sentence in self.knowledge:
        self.handle_mines(sentence)
        self.handle_safes(sentence)

    for i in range(len(self.knowledge)):
        for j in range(len(self.knowledge)):
            sentence = self.knowledge[i]
            other_sentence = self.knowledge[j]

            if other_sentence.cells <= sentence.cells:
                if new_cells := sentence.cells - other_sentence.cells:
                    new_count = sentence.count - other_sentence.count
                    new_sentence = Sentence(new_cells, new_count)
                    self.knowledge.append(new_sentence)

                    self.handle_mines(new_sentence)
                    self.handle_safes(new_sentence)

        self.remove_duplicates()
        self.remove_empty_sentences()

    def remove_duplicates(self):
        unique_sentences = []
        seen_cells = set()

        for sentence in self.knowledge:
            key = (tuple(sorted(sentence.cells)), sentence.count)

```

```

        if key not in seen_cells:
            unique_sentences.append(sentence)
            seen_cells.add(key)

    self.knowledge = unique_sentences

def remove_empty_sentences(self):
    for sentence in self.knowledge:
        if not sentence.cells and not sentence.count:
            self.knowledge.remove(sentence)

def handle_mines(self, sentence):
    if mines := sentence.known_mines():
        for mine in mines:
            self.mines.add(mine)
            for other in self.knowledge:
                other.mark_mine(mine)

    if self.mines:
        for cell in sentence.cells.copy():
            if cell in self.mines:
                sentence.mark_mine(cell)

def handle_safes(self, sentence):
    if safes := sentence.known_safes():
        for safe in safes:
            self.safes.add(safe)
            for other in self.knowledge:
                other.mark_safe(safe)

    if self.safes:
        sentence.cells -= self.safes

def neighboring_cells(self, cell):
    """
    Returns the list of neighboring cells given a cell.
    """
    i, j = cell

```

```

il = [i + k for k in range(-1, 2) if 0 <= i + k <= self.height - 1]
jl = [j + k for k in range(-1, 2) if 0 <= j + k <= self.width - 1]
return {(x, y) for x in il for y in jl if (x, y) != cell}

def make_safe_move(self):
    """
    Returns a safe cell to choose on the Minesweeper board.
    The move must be known to be safe, and not already a move
    that has been made.

    This function may use the knowledge in self.mines, self.safes
    and self.moves_made, but should not modify any of those values.
    """
    if safe_moves := (self.safes - self.moves_made):
        choice = random.choice(list(safe_moves))
        return choice
    return None

def make_random_move(self):
    """
    Returns a move to make on the Minesweeper board.
    Should choose randomly among cells that:
        1) have not already been chosen, and
        2) are not known to be mines
    """
    all_moves = {(i, j) for i in range(self.height) for j in range(self.width)}

    if remaining_moves := list(all_moves - self.moves_made - self.mines):
        choice = random.choice(remaining_moves)
        return choice
    return None

```

Implementation and Decision-Making Mechanism

The implementation of the Minesweeper AI involves several key steps that integrate decision-making techniques to enable the AI to play effectively.

1. Game Board Initialization:

- The `Minesweeper` class initializes the game board with specified dimensions and randomly places mines.

2. Game Logic:

- Methods are implemented to check for mines, count nearby mines, and determine if the player has won.

3. Knowledge Representation:

- The `Sentence` class tracks cells and the number of mines in those cells, facilitating reasoning. The AI uses logical sentences to represent its knowledge about the game state.

4. AI Decision-Making:

- The `MinesweeperAI` class manages interactions with the game. It marks cells as safe or mines and makes move decisions based on its knowledge.
- The AI employs logical inference to deduce safe moves and mine locations by updating its knowledge base when cells are revealed.

5. Decision Strategies:

- The AI prioritizes making safe moves whenever possible. It first checks its knowledge base for known safe cells and selects one to reveal. If no safe moves are available, it randomly selects a cell that is not known to be a mine.
- Heuristics are used to evaluate potential moves based on the density of mines in neighboring cells, helping the AI make more strategic decisions.

6. Dynamic Updating:

- After each move, the AI dynamically updates its knowledge base, refining its decision-making process as the game progresses. This continuous learning enables the AI to adapt to the changing game state.

Testing and Results

To test the game, a testing script named `runner.py` is created. This script initializes the game environment and runs a series of games with varying board sizes and mine densities. This allows for a robust assessment of the AI's decision-making capabilities under different conditions.

```

import pygame
import sys
import time

from minesweeper import Minesweeper, MinesweeperAI

HEIGHT = 10
WIDTH = 10
MINES = 10

# Colors
BLACK = (0, 0, 0)
GRAY = (180,180,179)
WHITE = (255, 255, 255)

# Create game
pygame.init()
size = width, height = 600, 400
screen = pygame.display.set_mode(size)
image = pygame.image.load('assets/images/background.jpg').convert()

# Fonts
OPEN_SANS = "assets/fonts/OpenSans-Regular.ttf"
smallFont = pygame.font.Font(OPEN_SANS, 20)
mediumFont = pygame.font.Font(OPEN_SANS, 28)
largeFont = pygame.font.Font(OPEN_SANS, 40)

# Compute board size
BOARD_PADDING = 20
board_width = ((2 / 3) * width) - (BOARD_PADDING * 2)
board_height = height - (BOARD_PADDING * 2)
cell_size = int(min(board_width / WIDTH, board_height / HEIGHT))
board_origin = (BOARD_PADDING, BOARD_PADDING)

# Add images
flag = pygame.image.load("assets/images/flag.png")
flag = pygame.transform.scale(flag, (cell_size, cell_size))
mine = pygame.image.load("assets/images/mine.png")

```



```

mine = pygame.transform.scale(mine, (cell_size, cell_size))

# Create game and AI agent
game = Minesweeper(height=HEIGHT, width=WIDTH, mines=MINES)
ai = MinesweeperAI(height=HEIGHT, width=WIDTH)

# Keep track of revealed cells, flagged cells, and if a mine was hit
revealed = set()
flags = set()
lost = False

# Show instructions initially
instructions = True

while True:

    # Check if game quit
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            sys.exit()

    screen.blit(image, (0, 0))

    # Show game instructions
    if instructions:

        # Title
        title = largeFont.render("Play Minesweeper", True, WHITE)
        titleRect = title.get_rect()
        titleRect.center = ((width / 2), 50)
        screen.blit(title, titleRect)

        # Rules
        rules = [
            "Click a cell to reveal it.",
            "Right-click a cell to mark it as a mine.",
            "Mark all mines successfully to win!"
        ]

```

```

for i, rule in enumerate(rules):
    line = smallFont.render(rule, True, WHITE)
    lineRect = line.get_rect()
    lineRect.center = ((width / 2), 150 + 30 * i)
    screen.blit(line, lineRect)

# Play game button
buttonRect = pygame.Rect((width / 4), (3 / 4) * height, width / 2, 50)
buttonText = mediumFont.render("Play Game", True, BLACK)
buttonTextRect = buttonText.get_rect()
buttonTextRect.center = buttonRect.center
pygame.draw.rect(screen, WHITE, buttonRect)
screen.blit(buttonText, buttonTextRect)

# Check if play button clicked
click, _, _ = pygame.mouse.get_pressed()
if click == 1:
    mouse = pygame.mouse.get_pos()
    if buttonRect.collidepoint(mouse):
        instructions = False
        time.sleep(0.3)

pygame.display.flip()
continue

# Draw board
cells = []
for i in range(HEIGHT):
    row = []
    for j in range(WIDTH):

        # Draw rectangle for cell
        rect = pygame.Rect(
            board_origin[0] + j * cell_size,
            board_origin[1] + i * cell_size,
            cell_size, cell_size
        )
        pygame.draw.rect(screen, GRAY, rect)

```

```

pygame.draw.rect(screen, BLACK, rect, 3)

# Add a mine, flag, or number if needed
if game.is_mine((i, j)) and lost:
    screen.blit(mine, rect)
elif (i, j) in flags:
    screen.blit(flag, rect)
elif (i, j) in revealed:
    neighbors = smallFont.render(
        str(game.nearby_mines((i, j))),
        True, BLACK
    )
    neighborsTextRect = neighbors.get_rect()
    neighborsTextRect.center = rect.center
    screen.blit(neighbors, neighborsTextRect)

row.append(rect)
cells.append(row)

# AI Move button
aiButton = pygame.Rect(
    (2 / 3) * width + BOARD_PADDING, (1 / 3) * height - 50,
    (width / 3) - BOARD_PADDING * 2, 50
)
buttonText = mediumFont.render("AI Move", True, BLACK)
buttonRect = buttonText.get_rect()
buttonRect.center = aiButton.center
pygame.draw.rect(screen, WHITE, aiButton)
screen.blit(buttonText, buttonRect)

# Reset button
resetButton = pygame.Rect(
    (2 / 3) * width + BOARD_PADDING, (1 / 3) * height + 20,
    (width / 3) - BOARD_PADDING * 2, 50
)
buttonText = mediumFont.render("Reset", True, BLACK)
buttonRect = buttonText.get_rect()
buttonRect.center = resetButton.center

```

```

pygame.draw.rect(screen, WHITE, resetButton)
screen.blit(buttonText, buttonRect)

# Display text
text = "Lost" if lost else "Won" if game.mines == flags else ""
text = mediumFont.render(text, True, WHITE)
textRect = text.get_rect()
textRect.center = ((5 / 6) * width, (2 / 3) * height)
screen.blit(text, textRect)

move = None

left, _, right = pygame.mouse.get_pressed()

# Check for a right-click to toggle flagging
if right == 1 and not lost:
    mouse = pygame.mouse.get_pos()
    for i in range(HEIGHT):
        for j in range(WIDTH):
            if cells[i][j].collidepoint(mouse) and (i, j) not in revealed:
                if (i, j) in flags:
                    flags.remove((i, j))
                else:
                    flags.add((i, j))
            time.sleep(0.2)

elif left == 1:
    mouse = pygame.mouse.get_pos()

    # If AI button clicked, make an AI move
    if aiButton.collidepoint(mouse) and not lost:
        move = ai.make_safe_move()
        if move is None:
            move = ai.make_random_move()
            if move is None:
                flags = ai.mines.copy()
                print("No moves left to make.")
            else:

```

```

        print("No known safe moves, AI making random move.")
    else:
        print("AI making safe move.")
        time.sleep(0.2)

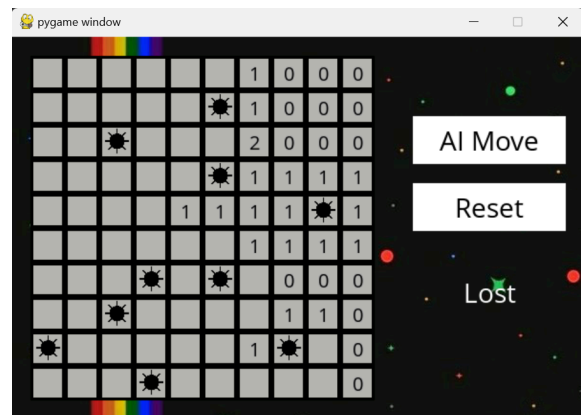
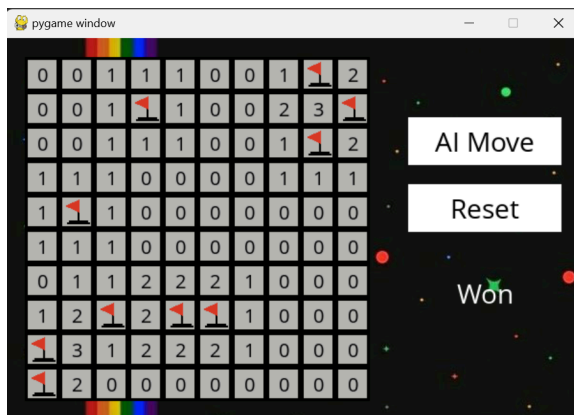
# Reset game state
elif resetButton.collidepoint(mouse):
    game = Minesweeper(height=HEIGHT, width=WIDTH, mines=MINES)
    ai = MinesweeperAI(height=HEIGHT, width=WIDTH)
    revealed = set()
    flags = set()
    lost = False
    continue

# User-made move
elif not lost:
    for i in range(HEIGHT):
        for j in range(WIDTH):
            if (cells[i][j].collidepoint(mouse)
                and (i, j) not in flags
                and (i, j) not in revealed):
                move = (i, j)

# Make move and update AI knowledge
if move:
    if game.is_mine(move):
        lost = True
    else:
        nearby = game.nearby_mines(move)
        revealed.add(move)
        ai.add_knowledge(move, nearby)

pygame.display.flip()

```



Conclusion

The Minesweeper AI project successfully demonstrates the application of logical reasoning and decision-making techniques in a classic game environment. By structuring the project around key classes and implementing a robust decision-making mechanism, the AI is capable of effectively navigating the game board, making informed choices, and improving its performance over time. Future work could explore enhancements to the AI's learning capabilities and further optimization of its decision-making strategies.